Technical Deployment Documentation: 2-Tier Flask-MySQL Application

Project: Flask Web Application Deployment

Organization: Modern India Pvt Limited

Prepared by: Vraj Lalwala – Technical Intern

Date: February 24, 2026

1. Executive Summary

This project involved the end-to-end deployment of a 2-tier web application consisting of a Flask frontend and a MySQL database. The solution was architected on AWS using Docker for containerization, ensuring a secure, scalable, and isolated environment.

2. System Architecture

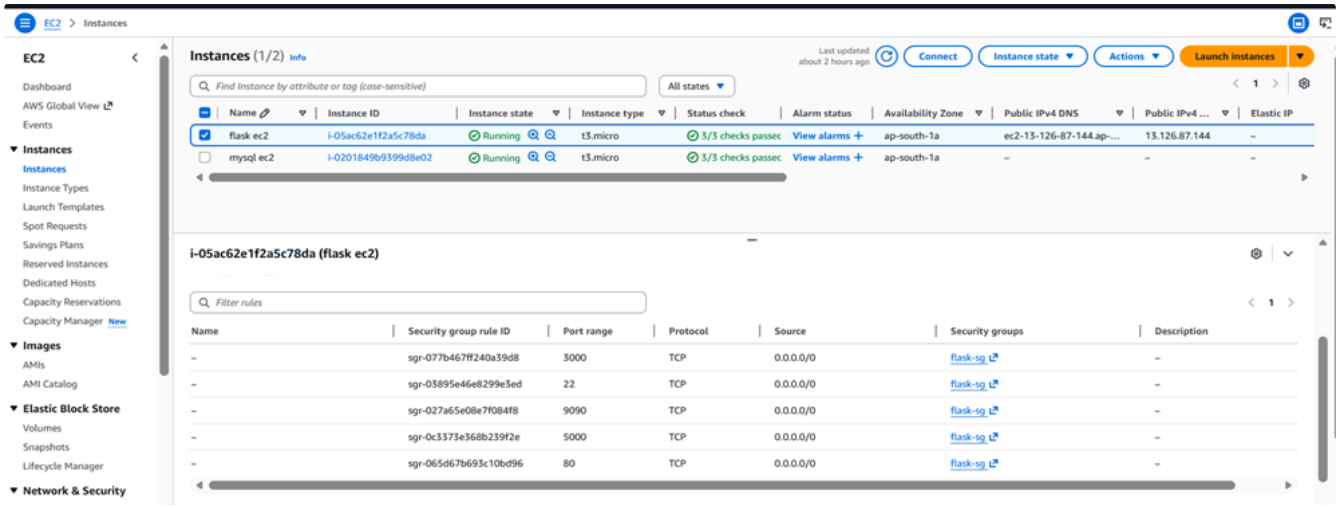
The application follows a standard 2-tier pattern:

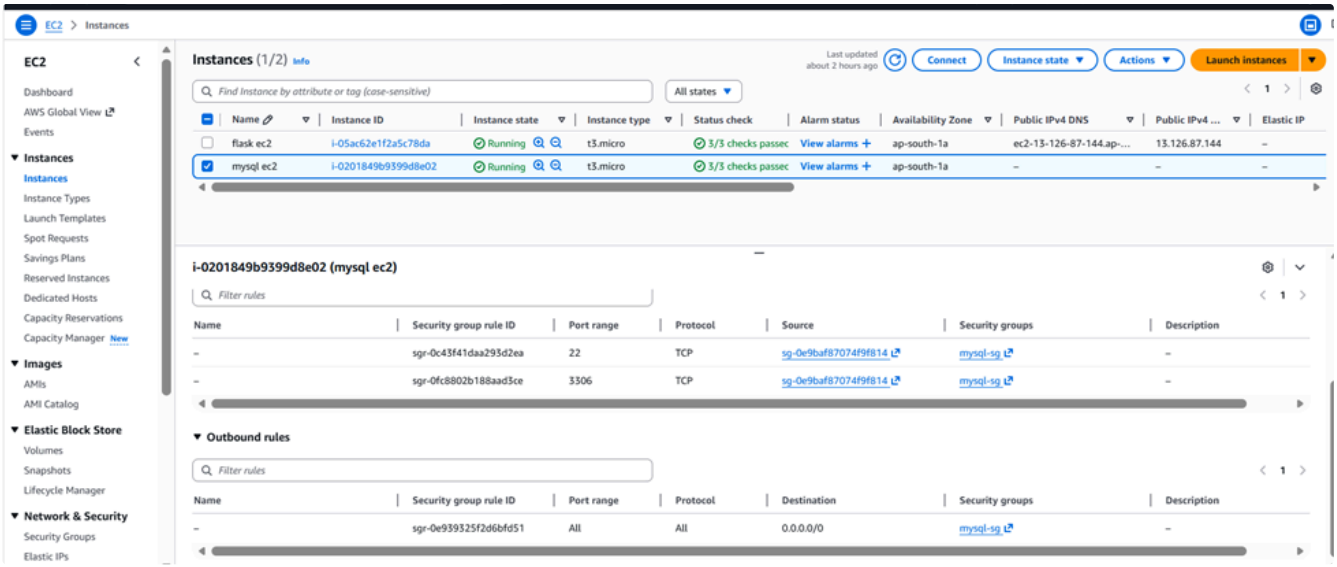
- Tier 1 (Application): Flask web server running in a Docker container, accessible via Port 5000 (mapped to Port 80).
- Tier 2 (Database): MySQL instance running in a Docker container, isolated from public access.

3. Infrastructure & Networking Setup

3.1 AWS EC2 Configuration

- Instance Type: t3.micro (Ubuntu 22.04 LTS)
- Security Group Rules:
  - Inbound: Port 22 (SSH), Port 80 (HTTP).
  - Outbound: All traffic (to allow package downloads and Docker Hub access).





4. **Application Deployment Strategy**

4.1 Containerization with Docker

I used Docker Compose to orchestrate the multi-container setup. This ensures that the Flask application and the MySQL database can communicate over a private bridge network using service names rather than unstable IP addresses.

4.2 Deployment Steps

1. Clone Repository: git clone GitHub - pushkar-iamops/2teir-flask
2. Environment Configuration: Created an .env file to store database credentials securely.
3. Build and Launch: Executed docker-compose up --build -d.

4.3 Docker Status Validation

Verified that both containers are running and healthy.

```
ubuntu@ip-10-0-1-210:~/2teir-flask/monitoring$ docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
          NAMES
d971e6f735a6   prom/prometheus       "/bin/prometheus --c…"   53 minutes ago   Up 53 minutes   0.0.0.0:9090->9090/tcp, [::]:9090->909
0/tcp   prometheus
6bed43c3c29c   prom/node-exporter    "/bin/node_exporter"     53 minutes ago   Up 53 minutes   0.0.0.0:9100->9100/tcp, [::]:9100->910
0/tcp   node-exporter
fe831ed9f78b   grafana/grafana       "/run.sh"                53 minutes ago   Up 53 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->300
0/tcp   grafana
03cac87adaf8   2teir-flask_backend   "python app.py"          58 minutes ago   Up 58 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->500
0/tcp   flask-backend
ubuntu@ip-10-0-1-210:~/2teir-flask/monitoring$
```

---

5. **Integration & Database Validation**

5.1 Database Connectivity

The Flask application was configured to connect to the MySQL container using the environment variables defined during the build.

5.2 Data Persistence Check

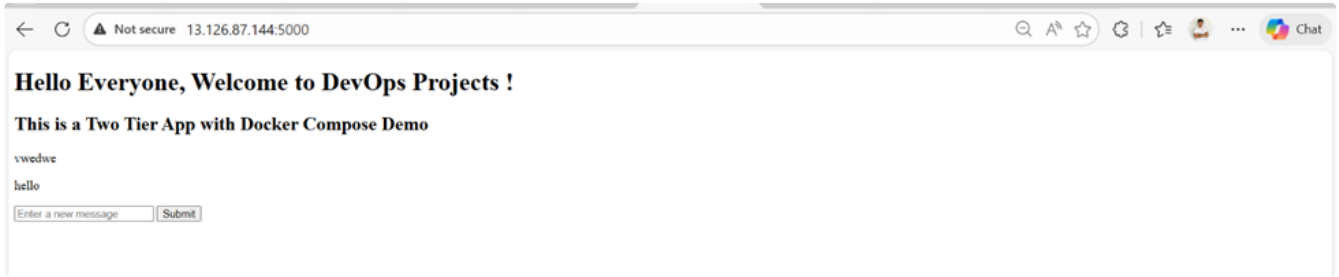
I accessed the MySQL container to verify that the tables were created successfully and that the application can write data.

```
ubuntu@ip-10-0-2-9:~$ sudo systemctl restart mysql
ubuntu@ip-10-0-2-9:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE two_tier;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> CREATE USER 'flaskuser'@'%' IDENTIFIED BY 'test@123';
ERROR 1819 (HY000): Your password does not satisfy the current policy requirements
mysql>
mysql> GRANT ALL PRIVILEGES ON two_tier.* TO 'flaskuser'@'%';
ERROR 1410 (42000): You are not allowed to create a user with GRANT
mysql>
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> USE two_tier;
Database changed
mysql>
mysql> CREATE TABLE messages (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   message TEXT
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE USER 'flaskuser'@'%' IDENTIFIED BY 'Test@12345';
Query OK, 0 rows affected (0.01 sec)
```

6. Final Application Delivery

6.1 Web Interface Access

The application is now live and reachable via the EC2 Public IP.



---

7. Monitoring & Observability

7.1 Metrics Collection

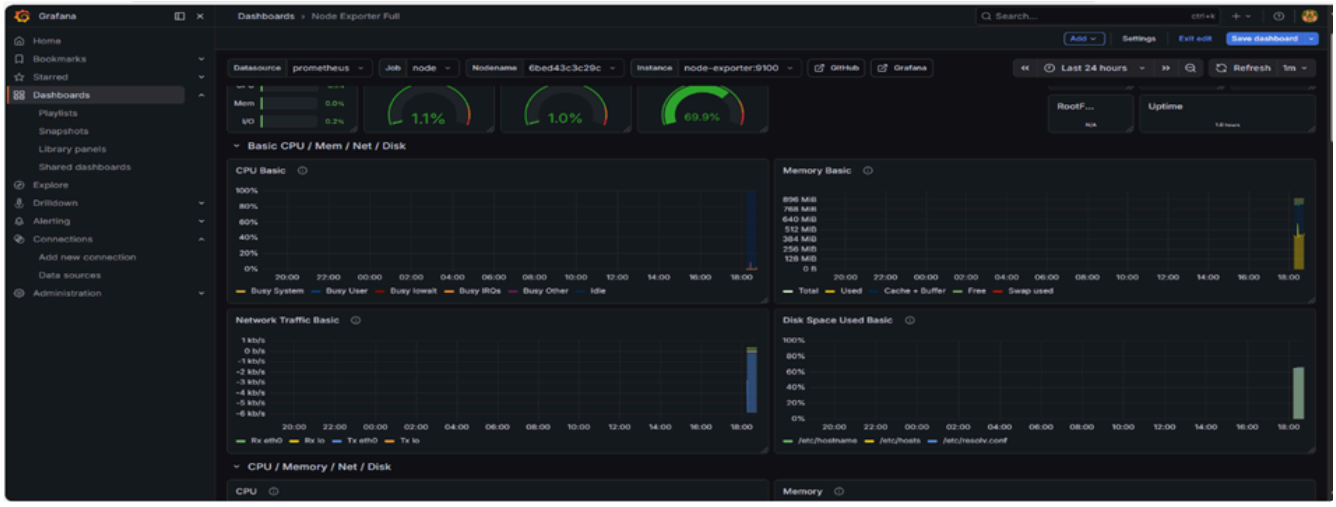
- **Prometheus:** Configured to scrape metrics from the Flask application and the MySQL exporter.

- **Grafana:** Integrated Prometheus as the primary data source to visualize system health.

7.2 Dashboards

I implemented dashboards to monitor:

- **EC2 Health:** CPU usage and Memory pressure.
- **App Performance:** HTTP request volume and response times.
- **Database Metrics:** Active connections and buffer pool usage.



---

8. **Security & Best Practices Implemented**

1. **Network Isolation:** MySQL is **not** exposed to the internet. Only the Flask container can reach the DB on Port 3306.
2. **Credential Management:** No passwords were hardcoded in the source code; all secrets were handled via environment variables.
3. **Least Privilege:** Security groups were restricted to only necessary ports (80, 22).

9. **Conclusion:-**
   The 2-tier Flask application has been successfully deployed following Modern India Pvt Limited's requirements. The use of Docker ensures that the deployment is repeatable and the environment is consistent.